

pw-27th nov-6

—using text content method instead of all text content methods.

```
test("handle auto suggestion without alltextcontents", async({page})=>{

  //goto page
  await page.goto("https://www.google.com/")

  await page.pause()

  //search in google search box with delay.
  await page.locator("//textarea[@name='q']").type("mukesh otwani", {delay:2000})

  //captures all the values seen on dom in variable.
  const elements=await page.locator("//div[@role='option']")

  //get the count of elements
  const count=await elements.count()

  //log the count
  console.log("total count " + count)

  //for loop to iterate.
  for(let i=0;i<count;i++){

    //get the value one by one
    const textvalue=await elements.nth(i).textContent()

    //print the value
    console.log(textvalue)

    //if text value is api click on it and break
    if(textvalue.includes("api")){
      elements.nth(i).click()
      break;
    }
  }

  //validate if title of page contains api
  await expect(page).toHaveTitle(/.*api/)

})
```

—grouping

this is how we add tags like smoke, regression etc. use @ followed by group name.

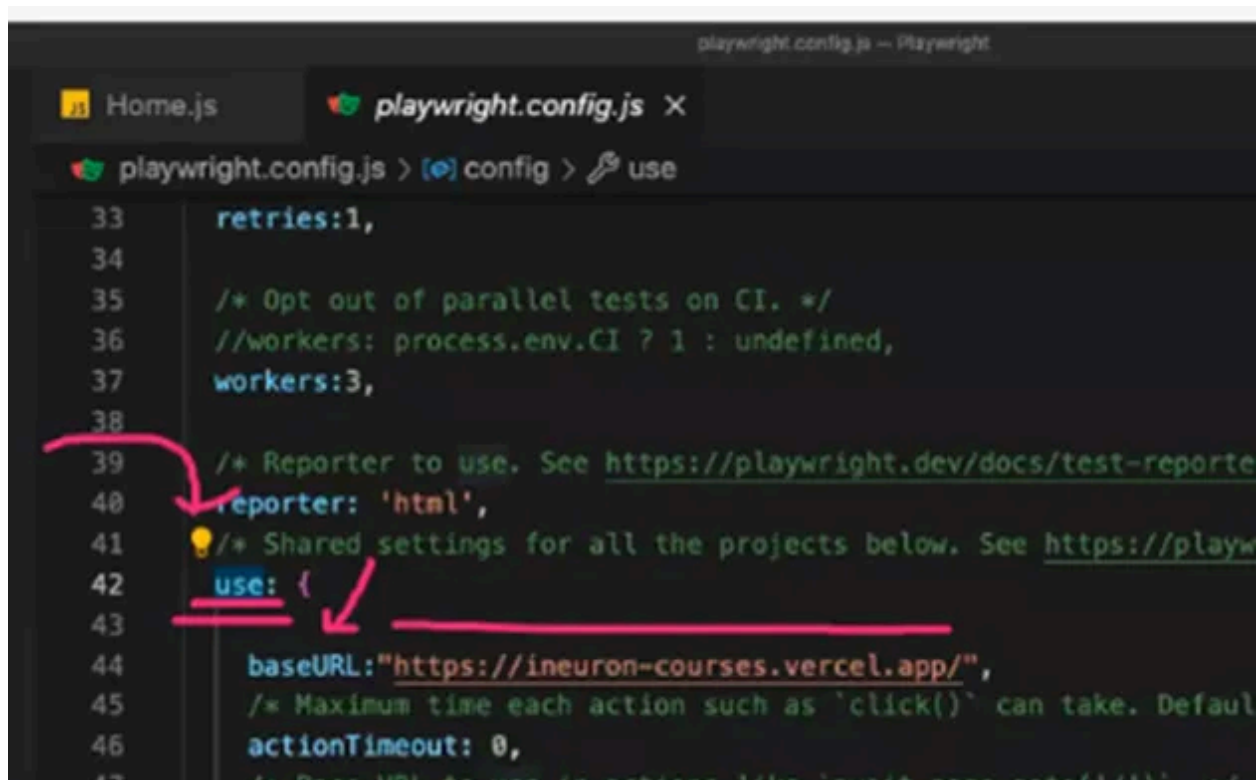
```
test("@regression handle window or tabs", async({browser})=>{
```

how to run the groups from cmd-

```
npx playwright test --grep "@Smoke"
```

—

Use pw config to set base urls.



—to hover over element-

```
await this.manage.nth(0).hover()
```

—to wait for some time

```
await this.page.waitForTimeout(2000)
```

—if we want to move the focus to the webpage after hover, clicks anywhere on webpage.

await this.page.locator("html").click()

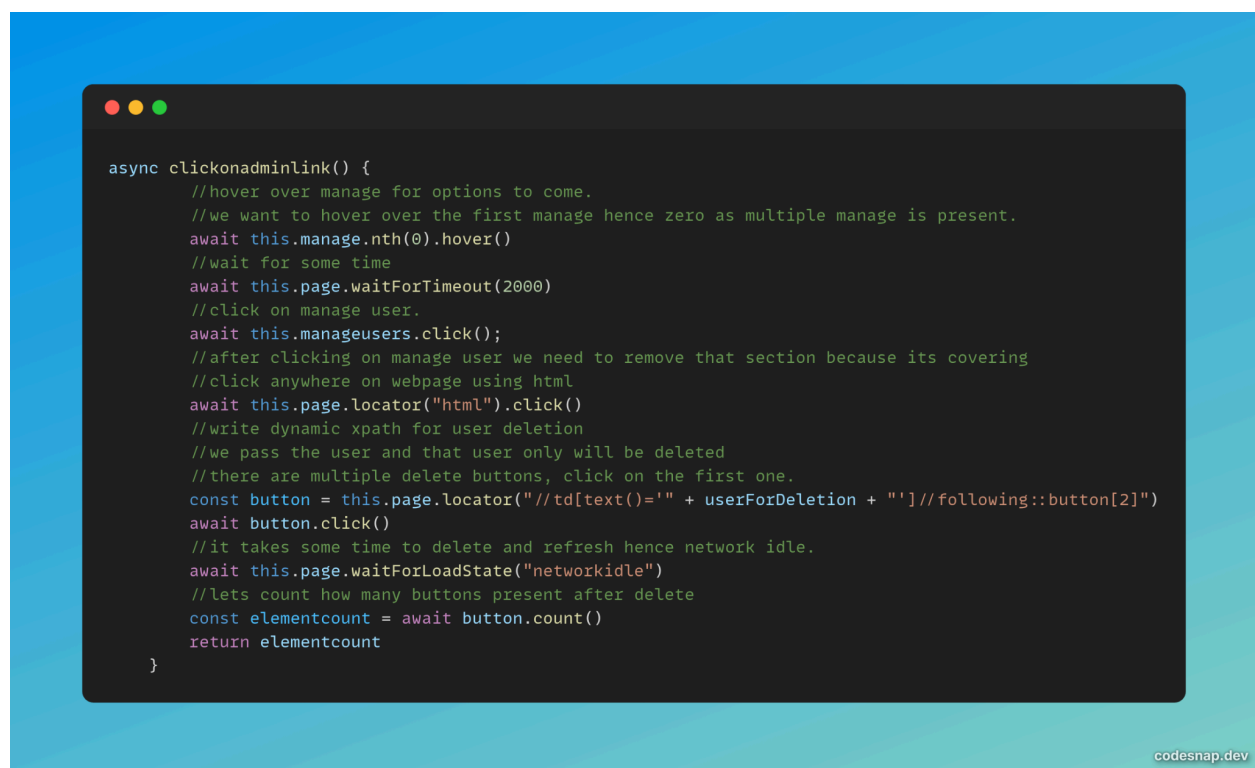
—dynamic xpath for capturing all buttons example-



—wait for network idle

await this.page.waitForLoadState("networkidle")

—see how to write function with return inside class-



—sample pom example

```

//dashboard page.js
//You wont get pw auto suggestion in normal js page files.
//We can have locators in const or in methods.

//first create class
class dashboard {

    //constructor of class
    //pass in page context
    //manage link locator
    //manage users locator.
    constructor(page) {
        this.page = page;
        this.adminlink = page.locator("//span[@class='oxd-text oxd-text--span oxd-main-menu-item--name'][normalize-space()='Admin']")
    }

    async clickonadminlink() {
        this.adminlink.click()
    }
}

//use the class name here.
module.exports = dashboard;

```

codesnap.dev

```

//login page.js-

class login{

    //in constructor keep only fixtures not method parameters
    //so we separated out the method for username and password.
    constructor(page){
        //without this keyword on the page. this.page=page.we get errors like
        //TypeError: Cannot read properties of undefined (reading 'goto')
        this.page=page;
        this.username=page.getByPlaceholder("Username")
        this.password=page.getByPlaceholder("Password")
        // this.signinbutton=page.getByText("Sign in")
        this.signinbutton=page.getByRole('button', { name: ' Login ' })
    }

    //login using username and password
    async logintoapplication(username, password){
        await this.page.pause()
        await this.page.goto("https://opensource-demo.orangehrmlive.com/web/index.php/auth/login")
        // await this.page.goto("https://www.opensource-demo.orangehrmlive.com/web/index.php/auth/login")
        await this.username.type(username)
        await this.password.type(password)
        //click on first sign in button
        // await this.signinbutton.nth(1).click()
        await this.signinbutton.click()
    }
}

module.exports=login;

```

codesnap.dev

```

//admin home page .js-
class adminpage {

  constructor(page) {
    this.page = page;
    this.addbutton = page.getByText("Add")
    this.firstselectdropdown=page.getByText('-- Select --')
    this.secondselectdropdown=page.getByText('-- Select --')
    this.employeenname = page.getByPlaceholder("Type for hints...")
    this.username = page.getByText("Username")
    this.password = page.locator("//input[@type='password']")[1]
    this.confirmpassword = page.getByText("Confirm Password")
    this.savebutton=page.locator("//button[normalize-space()='Save']")
  }

  async clickonaddbutton() {
    await this.addbutton.click()
  }

  async enterthedetails(userrole, empname, status, username, password, confirmpassword) {

    await this.firstselectdropdown.first().click();
    //smart pw doesnt need like crazy java all concat strings for dynamic xpaths.
    //directly specify the variable name.
    await this.page.getByRole('option', { name: userrole }).click();
    await this.secondselectdropdown.click();
    await this.page.getByRole('option', { name: status}).click();
    await this.employeenname.click()
    await this.employeenname.fill(empname)
    await this.page.waitForTimeout(2000)
    // await this.employeenname.getByText('t').click();

    await this.username.fill(username)
    await this.password.fill(password)
    await this.confirmpassword.fill(confirmpassword)

    await this.page.pause()

    await this.savebutton.click()

  }
}

module.exports=adminpage;

```

codesnap.dev

-test class -

```

// add user from admin pom spec
//first is import
import { test, expect } from "@playwright/test"

//then all page class to be imported
const dashboardpage = require("../pages/dashboardpage")
const adminhomepage = require("../pages/adminhomepage")
const loginpage = require("../pages/loginpage")

//add a user from admin
test("add user from admin", async ({ page }) => {
  //create objects of the classes
  //we need to use the variables above with require for new objects not the original class name
  const home = new adminhomepage(page)
  const login = new loginpage(page)
  const dashboard = new dashboardpage(page)

  //open the application
  await login.logintoapplication("Admin", "admin123")
  //check if the url contains dashboard word
  await expect(page).toHaveURL(/.*dashboard/)
})

//add user now
test("add user from admin page", async ({ page }) => {

  const home = new adminhomepage(page)
  const login = new loginpage(page)
  const dashboard = new dashboardpage(page)

  await dashboard.clickonadminlink()

  await login.logintoapplication("Admin", "admin123")

  await home.clickonaddbutton()

  await home.enterthedetails("ESS", "t", "Enabled", "tester123457", "tester123456", "tester123456")

  // Now check that the Reset button is visible
  const resetButton = await page.getByRole('button', { name: 'Reset' });
  await expect(resetButton).toBeVisible();

})

```

codesnap.dev

—to run specific scripts.

add them in scripts in this format.

```
5   "main": "index.js",  
    > Debug  
6   "scripts": {  
7     "ddt": "npx playwright test ./tests/deleteUser.spec.js --project=chromium",  
8     "allTestInChrome": "npx playwright test --project=chromium",  
9     "allTestInFirefox": "npx playwright test --project=firefox",  
10    "allTestInWebKit": "npx playwright test --project=webkit"  
11  },
```

pass the key in cmd.

```
mukeshotwani@Mukeshs-MacBook-Air-2 Playwright % npm run ddt
```

—my own awesome code to add user in orange hrm-

```

//adding home page class-
class adminpage {

  constructor(page) {
    this.page = page;
    this.addbutton = page.getByText("Add")
    this.firstselectdropdown=page.getByText('-- Select --')
    this.secondselectdropdown=page.getByText('-- Select --')
    this.employeeenane = page.getByPlaceholder("Type for hints...")
    this.username = page.locator("//input[@class='oxd-input oxd-input--active']")[2]
    this.password = page.locator("//input[@type='password']")[1]
    this.confirmpassword = page.locator("//input[@type='password']")[2]
    this.savebutton=page.locator("//button[normalize-space()='Save']")
  }

  async clickonaddbutton() {
    await this.addbutton.click()
  }

  async enterthedetails(userrole, empname, status, username, password, confirmpassword) {

    await this.firstselectdropdown.first().click();
    //smart pw doesnt need like crazy java all concat strings for dynamic xpath.
    //directly specify the variable name.
    await this.page.getByRole('option', { name: userrole }).click();
    await this.secondselectdropdown.click();
    await this.page.getByRole('option', { name: status}).click();
    await this.employeeenane.click()
    await this.employeeenane.fill(empname)
    // Use keyboard to wait for and select the first suggestion
    await this.page.waitForTimeout(1000); // Wait briefly for dropdown to populate (adjust if needed)
    await this.page.keyboard.press('ArrowDown'); // Select first option
    await this.page.keyboard.press('Enter'); // Confirm selection
    // await this.page.waitForTimeout(2000)
    // await this.employeeenane.getByText(empname).click();

    // await this.employeeenane.getByText('t').click();

    await this.username.fill(username)
    await this.password.fill(password)
    await this.confirmpassword.fill(confirmpassword)

    await this.page.pause()

    await this.savebutton.click()

  }
}
module.exports=adminpage;

```



```
//dashboard page class
//You wont get pw auto suggestion in normal js page files.
//We can have locators in const or in methods.

//first create class
class dashboard {

    //constructor of class
    //pass in page context
    //manage link locator
    //manage users locator.
    constructor(page) {
        this.page = page;
        this.adminlink = page.locator("//span[@class='oxd-text oxd-text--span oxd-main-menu-item--name'][normalize-space()='Admin']")
        this.getFirstusername=page.locator("//div[@role='cell'])[4]")
    }

    async clickonadminlink() {
        await this.adminlink.click()
        const usernametext= await this.getFirstusername.textContent();
        return usernametext;
    }
}

//use the class name here.
module.exports = dashboard;
```

```

//login page class

class login{

    //in constructor keep only fixtures not method parameters
    //so we separated out the method for username and password.
    constructor(page){
        //without this keyword on the page. this.page=page.we get errors like
        //TypeError: Cannot read properties of undefined (reading 'goto')
        this.page=page;
        this.username=page.getByPlaceholder("Username")
        this.password=page.getByPlaceholder("Password")
        // this.signinbutton=page.getByText("Sign in")
        this.signinbutton=page.getByRole('button', { name: ' Login ' })
    }

    //login using username and password
    async logintoapplication(username, password){
        await this.page.pause()
        await this.page.goto("https://opensource-demo.orangehrmlive.com/web/index.php/auth/login")
        // await this.page.goto("https://www.opensource-demo.orangehrmlive.com/web/index.php/auth/login")
        await this.username.type(username)
        await this.password.type(password)
        //click on first sign in button
        // await this.signinbutton.nth(1).click()
        await this.signinbutton.click()
    }
}

module.exports=login;

```

codesnap.dev

test method

```
//add user now
test("add user from admin page", async ({ page }) => {

  const home = new adminhomepage(page)
  const login = new loginpage(page)
  const dashboard = new dashboardpage(page)

  await login.logintoapplication("Admin", "admin123")

  let usernametext= await dashboard.clickonadminlink()

  await home.clickonaddbutton()

  await home.enterthedetails("ESS", usernametext, "Enabled", "tester123458", "tester123456", "tester123456")

  // Now check that the Reset button is visible
  const resetButton = await page.getByRole('button', { name: 'Reset' });
  await expect(resetButton).toBeVisible();

})
```